u4lib: An AI-Ready Computational Framework for U(4) Supermultiplet Symmetry, Wigner Coupling, and Racah Recoupling Coefficients

Phong Dang,[1] Daniel Langr,[2] Tomas Dytrych,[3,1] and Michela Negro[1]

[1]*Department of Physics and Astronomy, Louisiana State University, Baton Rouge, LA 70803, USA*
[2]*Department of Computer Systems, Faculty of Information Technology, Czech Technical University in Prague, Prague, Czech Republic*
[3]*Nuclear Physics Institute, Czech Academy of Sciences, 250 68 Řež, Czech Republic*

Wigner's supermultiplet symmetry treats nucleon spin and isospin within a common U(4)/SU(4) description. The C++ library `u4lib` supplies the coupling coefficients and algebraic operations needed for calculations in this scheme. Its canonical Gelfand–Tsetlin basis and physical SU(2) × SU(2) basis support complementary descriptions of the same states, connected by an explicit basis transformation. Users can construct irreducible representations (irreps), obtain dimensions and Casimir invariants, conjugate states, enumerate bases and multiplicities, and decompose tensor products. Matrix-element routines cover the canonical U(4) generators and the physical spin **S**, isospin **T**, and Gamow–Teller-type tensor **U**. Wigner coupling and Racah recoupling coefficients are evaluated by the procedure of Dang *et al.* [1]. The implementation builds on `u3lib`, with a header-only C++ interface and corresponding Python bindings. A catalog of agent skills documents how an AI assistant can prepare, execute, and check these calculations through natural-language requests. This manual describes the public release and evaluates the use of those skills in coding tasks.

PROGRAM SUMMARY

Program title: `u4lib`
Licensing provisions: MIT License.
Code repository: https://github.com/Deep-SANE/u4lib
Programming language: C++ (C++20), with Python bindings via pybind11.
External libraries: `u3lib` (prerequisite, providing Eigen3, Boost headers, OpenMP, fmt, libwignernj, and core utilities); and a null-space solver backend — dense Eigen3 QR (the default) or `sns`. The Python extension additionally uses pybind11.
Nature of problem: Efficient, accurate, and reproducible computation of the Wigner coupling coefficients for the two group–subgroup chains SU(4) ⊃ U(3) (canonical) and SU(4) ⊃ SU(2) × SU(2) (physical, spin-isospin), the U(4) Racah recoupling coefficients (U-, Z-, and 9-coefficients), and the associated representation-theoretic quantities (dimensions, Casimirs, multiplicities, basis enumeration, generator matrix elements, and basis transformations) needed by symmetry-adapted nuclear-structure calculations.
Solution method: The coupling and recoupling coefficients are built from the highest-weight canonical reduced coefficients, obtained as the orthonormal null space of the linear system enforced by the raising generators, and propagated by the recursion relations of Ref. [1]; the physical SU(2) × SU(2)-coupled coefficients follow from the physical → canonical basis transformation.

I. INTRODUCTION

Spin and isospin are the starting point for Wigner's supermultiplet theory [2]. Introduced in 1937, the theory combines them in a single SU(4) classification. For nuclear-structure calculations, this provides a complement to the spatial symmetry of Elliott's SU(3) model [3, 4] and the symmetry-adapted shell-model methods developed from it [5, 6]. Applications include charge-exchange and Gamow–Teller strength, as well as studies of SU(4) symmetry restoration in heavy and superheavy nuclei [7].

Working in the physical spin-isospin basis requires U(4) coupling and recoupling coefficients. Their construction has been studied by Hecht and Pang [8], through Draayer's U(4) ⊃ SU(2) ⊗ SU(2) projection techniques [9], and by Partensky and Maguin [10] and Rowe and Repka [11]. More recent work by Pan and collaborators develops the physical $\mathrm{SU_S(2)} \otimes \mathrm{SU_T(2)}$ basis and its elementary Wigner coefficients [12–14]. Other available tools include general SU(N) Clebsch–Gordan algorithms [15] and a program for the canonical SU(4) chain [16].

The numerical method used here connects this work to developments in U(3). SU(3) coefficients have been available through the Draayer–Akiyama Fortran programs [17, 18] and subsequent refinements [19, 20]. Following the U(3) procedure of Ref. [21], Dang *et al.* developed the U(4) algorithm implemented in `u4lib` [1]. Raising-generator

equations and numerical null-space solvers replace the evaluation of hypergeometric sums. Reference [1] contains the derivation; the present manual focuses on the resulting library and its use.

Several other fields also use SU(4) representations. In the strong interaction, a spin–isospin SU(4) emerges in the large-N_c limit of QCD [22] and organizes the essential elements of nuclear binding seen in lattice calculations [23]. In quantum information, the two-qubit unitary group is SU(4), and the geometry of its nonlocal gates follows from a Cartan decomposition of the algebra [24]. In ultracold-atom physics, alkaline-earth atoms realize SU(N)-symmetric magnetism that includes SU(4) [25], while in condensed matter the spin–valley degrees of freedom of graphene form an approximate SU(4) quantum Hall ferromagnet [26]. These applications may use the representation data supplied by `u4lib`, although their physical models and basis conventions must be handled separately.

A calculation in `u4lib` can begin with either a canonical Gelfand–Tsetlin state in the U(4) $\supset$ U(3) $\supset$ U(2) $\supset$ U(1) chain or a physical state labeled by spin and isospin in SU(4) $\supset$ $\mathrm{SU_S}(2) \otimes \mathrm{SU_T}(2)$. The library supplies the transformation between these descriptions, generator matrix elements, Kronecker (tensor-product) decompositions, and irrep invariants alongside the coupling routines. Its dependence on `u3lib` reflects the canonical subgroup chain and allows the two libraries to share numerical infrastructure. Both use header-only C++ interfaces with matching Python bindings.

`u4lib` belongs to `DeepSANE`: **Deep**-learning and **S**ymmetry-adapted **A**b-initio **N**uclear modeling: an AI-ready **E**cosystem. Within this project, agent skills provide a practical entry point for users who want to describe a calculation in words. For U(4), such requests require care with representation labels, doubled spin and isospin, multiplicity indices, and phases. C++ programs must also respect buffer lifetimes and thread-local workspaces. The skills record these rules and attach numerical checks to the corresponding operations.

Keeping such instructions with the code is also being explored in broader scientific-software efforts, including the U.S. Department of Energy (DOE) Genesis Mission's scientific agent skill catalog[1]. Here we assess their usefulness through three coding tasks, measuring whether access to the catalog changes correctness, token use, runtime, and API cost.

Sections II and III describe the source layout and C++ reference interface. Section IV covers Python use, and Sec. V explains the skills and reports the coding experiments. Build instructions appear in Sec. VI.

II. LIBRARY STRUCTURE

The headers contain the U(4) implementation and use `u3lib` as a dependency. Examples, bindings, tests, and documentation occupy separate directories:

- `include/`: the library itself (all algorithms live here; user programs include these headers directly):
  - `U4irrep.h`: the irrep classes `su4::irrep`, `u4::irrep`;
  - `U4gelfand.h`, `internal/gelfand/`: the Gelfand state class `u4::gelfand`;
  - `U4basis.h`: basis enumeration (`gelfandBasis`, `physicalBasis`), the inner multiplicity `Zeta`, the `u4::physicalvector` class, and the generator-action predicates;
  - `U4decomposition.h`: the outer multiplicity `Eta` and the tensor-product `decomposition`;
  - `U4U3xU4U3.h`, `U4SU2SU2xU4SU2SU2.h`: the canonical and physical coupling coefficients and their databases;
  - `U4_U6Z6Coeffs.h`, `U4_U9Coeffs.h`: the U-, Z-, and 9-coefficient recoupling coefficients and their databases;
  - `U4SU2SU2_to_U4U3U2U1.h`: the physical-to-canonical transformation;
  - `U4generatorsRME.h`: the physical-generator reduced matrix elements `RME_S`, `RME_T`, `RME_U`;
  - `Definitions.h`: the structured coefficient containers;
  - `u4lib.h`: the umbrella header and the `initialize`/`finalize` lifecycle.
- `src/`: small example driver programs (one `.cpp` per task) that call the headers and print results; compiled to `bin/`.
- `python/`: the pybind11 extension (`pyu4lib.cpp` → `u4lib.so`), its build (`make python`, `build.sh`), build notes (`BUILD_NOTES.md`), and a tutorial notebook (`tutorial.ipynb`).

[1] See the DOE Genesis Mission skill catalog: https://gitlab.osti.gov/genesis/genesis-skills.

Header	Entry point	Task
`U4irrep.h`	`dim`, `C1/C2/C3/C4`, `conjugate`	irrep dimension, Casimirs, conjugation
`U4basis.h`	`Zeta`, `gelfandBasis`, `physicalBasis`	inner (spin-isospin) multiplicity; basis enumeration
`U4decomposition.h`	`Eta`, `decomposition`	outer multiplicity; $A \otimes B$ series
`U4U3xU4U3.h`	`wigU4U3db`	SU(4)⊃U(3) canonical reduced CG
`U4SU2SU2xU4SU2SU2.h`	`wigU4SU2SU2db`	SU(2)×SU(2) physical CG
`U4_U6Z6Coeffs.h`	`u4u6db`, `u4z6db`	U- and Z-recoupling (three irreps)
`U4_U9Coeffs.h`	`u4u9db`	9-coefficient recoupling (four irreps)
`U4SU2SU2_to_U4U3U2U1.h`	`phys_to_cano`	physical-to-canonical transformation
`U4generatorsRME.h`	`RME_S`, `RME_T`, `RME_U`	reduced matrix elements of **S**, **T**, **U**

TABLE I: Main entry points of `u4lib`.

- `agents/`: the AI-agent skill catalog: an index (`AGENTS.md`), shared runtime instructions, and one skill per capability.
- `test/`: the regression suite (`test_*.cpp`, the `*_cases.h` case sets, and `reference/` documented values), run by `make test`.
- `doc/`: this manual (LaTeX source).
- `bin/`: build output (not source).

III. CLASSES AND LIBRARY INTERFACE

The interface separates algebraic objects from coefficient databases. Irreps, basis states, and their methods can be used immediately; database queries require the lifecycle calls below. U(4) types use the `u4` namespace and inherit SU(4) functionality from `su4`. Table I identifies the relevant header for each operation.

A. Lifecycle

In `u4lib.h`:

- `initialize()` / `initialize(&argc,&argv)`: set up the Wigner backend, the SO(3) caches, and the U(3) *and* U(4) coefficient databases (the cascade descends into `u3lib`);
- `finalize()`: release these resources in reverse order;
- `initialize_thread()` / `finalize_thread()`: per-thread setup and teardown for multithreaded use;
- `clear()`: discard cached coefficients while leaving the databases intact;
- `is_initialized()`: report the current state.

Shared initialization: `u4lib` initializes its `u3lib` dependency automatically. If the application also calls `u3lib` directly, initialize both libraries at startup and defer finalization until all coefficient calculations have finished. Their Wigner backend is shared; finalizing either library while the other still needs it causes runtime errors.

B. Irreps: `su4::irrep` and `u4::irrep`

(`U4irrep.h`.) `su4::irrep` stores an SU(4) irrep (P_1, P_2, P_3); `u4::irrep` derives from it and adds the fourth label n_{44} (fixing the total quanta).

Constructors:

- `su4::irrep()`: default (empty) SU(4) irrep.

- `su4::irrep(P1,P2,P3)`: SU(4) irrep from its labels.
- `u4::irrep()`: default (empty) U(4) irrep.
- `u4::irrep(n14,n24,n34,n44)`: U(4) irrep from its four labels.
- `u4::irrep(P1,P2,P3)`: normalized representative.
- `u4::irrep(su4::irrep, A)`: fix the total quanta A.

Label getters:

- `P1()`, `P2()`, `P3()`: the SU(4) labels.
- `n14()`, `n24()`, `n34()`, `n44()`: the four U(4) labels (only for `u4::irrep`).

Dimension and invariants:

- `dim()`: the irrep dimension (both classes).
- `C2()`, `C3()`, `C4()`: the SU(4) Casimirs of orders 2, 3, 4 (both classes, shift-invariant).
- `C1()`: the first-order (U(1)) Casimir = n_{14}+n_{24}+n_{34}+n_{44} (total quanta), on `u4::irrep` only (SU(4) is U(4) with the U(1) trace generator removed, so `su4::irrep` has no `C1()`).
- `quanta()`: synonym of `C1()`.

Other:

- `zero()`: test for the trivial irrep.
- `conjugate()`: the conjugate irrep (P_3, P_2, P_1).
- `conjugate_phase_power()`: the conjugation phase factor.
- `normalize()`: set $n_{44} = 0$ for `u4::irrep`.
- comparison operators and stream output.
- `writeBinary` / `readBinary`: serialization for binary files.

C. Gelfand states: `u4::gelfand`

(`U4gelfand.h`.) The canonical basis state with top U(4) labels $[n_{14}, n_{24}, n_{34}, n_{44}]$ and U(3)⊃U(2)⊃U(1) sublabels $[n_{13}, n_{23}, n_{33}]$, $[n_{12}, n_{22}]$, n_{11}.

Constructor:

- `u4::gelfand(n14,n24,n34,n44,n13,n23,n33,n12,n22,n11)`: build a Gelfand state from its labels.

Label getters and validity check:

- `n14()` ... `n11()`: the ten Gelfand-pattern labels.
- `check()`: test the betweenness condition.
- `normalize()`: shift all quantum numbers by n_{44}.

Subgroup labels:

- `P1()`, `P2()`, `P3()`: the SU(4) labels.
- `Lambdax2()`: 2× the embedded SU(2) angular momentum.
- `MLambdax2()`: 2× its projection.

Conjugation:

- `conjugate()`: the Gelfand state's conjugate.
- `conjugate_phase()`: the accompanying conjugation phase.

Generator matrix elements: The matrix elements of the U(4) canonical generators E_{ij} acting on a Gelfand state, returned by the methods `e11()` ... `e44()`, `e12`/`e21`, `e23_*`, `e13_*`, `e34_*`, `e24_*`, `e14_*` (and their lowering partners), where a suffix selects the target branch. The raising-action predicates `E12`, `E23_12`, ..., `E14_33_22` (in `U4basis.h`) identify the target state of each generator.

D. Physical states: `u4::physicalvector`

(`U4basis.h`.) The SU(2) × SU(2) reduced spin-isospin state $|(P_1P_2P_3)\,\zeta\,S\,T\rangle$. Spin and isospin are stored **doubled**: $SS = 2S$, $TT = 2T$.

Constructor:

- `u4::physicalvector(irrep, SS, TT)`: a physical state of doubled spin $SS = 2S$ and isospin $TT = 2T$.

Accessors:

- `irr()`: the underlying irrep.
- `P1()`, `P2()`, `P3()`: the SU(4) labels.
- `n14()` ... `n44()`: the U(4) labels.
- `SS()`, `TT()`: the doubled spin and isospin.
- `zeta()` / `zetamax()`: the inner multiplicity of (S, T).

E. Free functions

Multiplicities:

- `u4::Zeta(irrep,SS,TT)`: inner (spin-isospin) multiplicity $\zeta_{\max}$ of (S, T), returns 0 for labels outside the admissible grid.
- `u4::Eta(A,B,C)`: outer multiplicity $\eta_{\max}$ of irrep C in the $A \otimes B$ product (`U4decomposition.h`).

Bases:

- `u4::gelfandBasis(irrep)`: every Gelfand state of the irrep.
- `u4::physicalBasis(irrep)`: the physical states with nonzero ζ.
- `u4::findGelfand(g,gArr)`: the position of a state inside the `gArr` Gelfand basis.

Decomposition:

- `u4::decomposition(A,B)`: the complete Kronecker series, as a (list of C, list of $\eta_{\max}$) pair obeying $\sum_C \eta_{\max}(C) \dim C = \dim A \dim B$.

Physical generator reduced matrix elements: Each takes two `physicalvector` states (bra, ket) and two `std::vector<int>` out-parameters, and returns an `Eigen::MatrixXd` of shape $\zeta'_{\max} \times \zeta_{\max}$ (`U4generatorsRME.h`).

- `RME_S`, `RME_T`: diagonal in (S, T), with $\langle\|\mathbf{S}\|\rangle = \sqrt{S(S{+}1)(2S{+}1)(2T{+}1)}$ and $\langle\|\mathbf{T}\|\rangle = \sqrt{T(T{+}1)(2T{+}1)(2S{+}1)}$.
- `RME_U`: the rank-$(1, 1)$ Gamow–Teller-type tensor ($\Delta S, \Delta T \in \{-1, 0, +1\}$).

Transformation: Each routine expands the physical state `p` (at doubled projections MMS, MMT) over the Gelfand basis `gArr`, returning the canonical-state indices together with an amplitude matrix whose columns are orthonormal, one per ζ.

- `u4::phys_to_cano(p,MMS,MMT,gArr)`: the expansion at a single projection (M_S, M_T).
- `u4::phys_to_cano_allM(p,gArr)`: the expansion over all projections.

F. Coefficient containers

(`Definitions.h`.) The following containers hold coefficient blocks. Indices start at zero and must be smaller than the corresponding `*max()` count. Use `operator()` for coordinate access, `operator[]` for flat indexing, `data()` for the raw buffer, and `print()` and `operator<<`.

`U4U3`: canonical SU(4) $\supset$ U(3) coupling of $A \otimes B \to C$.

- `rhomax()`, `etamax()`: the multiplicity axes.
- `operator()(rho,eta)`.
- `data()`, `size()`, `empty()`.
- `print(s = std::cout)`.

`U4SU2SU2`: physical SU(2) $\times$ SU(2) coupling of $A \otimes B \to C$.

- `zetaAmax()`, `zetaBmax()`, `zetaCmax()`, `etamax()`: the multiplicity axes.
- `operator()(zetaA,zetaB,zetaC,eta)`.
- `data()`.
- `print(s = std::cout)`.

`U4U6`: U-recoupling.

- `etaABmax()`, `etaAB_Cmax()`, `etaBCmax()`, `etaA_BCmax()`: the multiplicity axes.
- `operator()(etaAB,etaAB_C,etaBC,etaA_BC)`.
- `data()`.
- `print(s = std::cout)`.

`U4Z6`: Z-recoupling.

- `rhoABmax()`, `rhoAB_Cmax()`, `rhoACmax()` (the AC channel), `rhoAC_Bmax()` (the AC_B channel): the multiplicity axes.
- `operator()(etaAB,etaAB_C,etaAC,etaAC_B)`.
- `data()`.
- `print(s = std::cout)`.

`U4U9`: 9-coefficient recoupling.

- `etaACmax()`, `etaBDmax()`, `etaABCDmax()`, `etaABmax()`, `etaCDmax()`, `etaACBDmax()`: the multiplicity axes.
- `operator()(etaAC,etaBD,etaABCD,etaAB,etaCD,etaACBD)`.
- `data()`.
- `print(s = std::cout)`.

Retain a copy of any C++ `U4U9` block that must survive the next query. This container references a thread-local scratch buffer, whose contents are replaced by the next database call. The other container types own their data.

G. Coefficient databases

The singleton accessors below retrieve coefficient blocks in the container formats just described.

- `u4::wigU4U3db().get_U4U3(` $n^a_{14}, n^a_{24}, n^a_{34}, n^a_{44}, n^a_{13}, n^a_{23}, n^a_{33},\ \ldots^b,\ \ldots^c$ `)` → `U4U3`: SU(4) ⊃ U(3) reduced coupling for $a \otimes b \to c$ (each state by its U(4) labels and U(3) sublabels), indexed by (ρ, η).
- `u4::wigU4SU2SU2db().get_U4SU2SU2(` $P^a_1, P^a_2, P^a_3, SS^a, TT^a,\ \ldots^b,\ \ldots^c$ `)` → `U4SU2SU2`: physical spin-isospin reduced coupling ($SS = 2S$, $TT = 2T$), indexed by $(\zeta_A, \zeta_B, \zeta_C, \eta)$.
- `u4::u4u6db().get_U4U6(` $A, B, ABC, C, AB,\ \eta_{AB}, \eta_{AB_C},\ BC, \eta_{BC}, \eta_{A_BC}$ `)` → `U4U6`: recouples $(AB)C \leftrightarrow A(BC)$ (note that ABC precedes C).
- `u4::u4z6db().get_U4Z6(` $B, A, ABC, C, AB,\ \eta_{AB}, \eta_{AB_C},\ AC, \eta_{AC}, \eta_{AC_B}$ `)` → `U4Z6`: recouples $(AB)C \leftrightarrow (AC)B$ (note the leading B, A, exchanging the first pair).
- `u4::u4u9db().get_U4U9(` $A, B, C, D,\ AB, CD, AC, BD, ABCD,\ \eta_{AC}, \eta_{BD}, \eta_{ABCD},\ \eta_{AB}, \eta_{CD}, \eta_{ACBD}$ `)` → `U4U9`.

The multiplicity maxima are the `Eta` values for the corresponding channels. Their positions are interleaved with the irrep arguments as shown above; an incorrect count triggers an internal assertion. In `get_U4U9`, note that the 3rd multiplicity argument η_{ABCD} corresponds to the $(AB)(CD) \to ABCD$ coupling route and the 6th (η_{ACBD}) to $(AC)(BD) \to ABCD$; exchanging these counts triggers an internal assertion. A forbidden or vanishing coupling yields an empty container, which is safe to inspect for its shape but not to index.

A minimal C++ session:

```
u4::initialize(&argc, &argv);
// (1,0,0) S=1/2 T=1/2  x  (1,0,0) S=1/2 T=1/2  ->  (2,0,0) S=1 T=1
auto blk = u4::wigU4SU2SU2db().get_U4SU2SU2(1,0,0,1,1, 1,0,0,1,1, 2,0,0,2,2);
double x = blk(0, 0, 0, 0);     // = 1
u4::finalize();
```

H. Conventions and normalization

Phase and normalization conventions follow Ref. [1]; documented values and the generation procedure are in `test/reference/`.

- **Doubled spin-isospin:** All spin and isospin labels are doubled: $SS = 2S$, $TT = 2T$ (and $MMS = 2M_S$, $MMT = 2M_T$). `Zeta` returns 0 off the parity-correct (S, T) grid; enumerate valid states with `physicalBasis` rather than scanning it.
- **Multiplicity indexing:** the `Zeta` and `Eta` routines report the counts, and every container accessor is indexed from zero up to the corresponding maximum $(0, \ldots, \max{-}1)$.
- **Canonical-coupling phase:** within each η block the highest-weight reduced coefficients set both phase and normalization; they span the orthonormal null space left once the raising generators annihilate the highest-weight coupled state. When $\eta_{\max} > 1$ (and for self-conjugate irreps) the solver returns one admissible orthonormal η basis, and all coefficients carrying η must use the same basis choice. Under conjugation a Gelfand state obeys $C\,|g\rangle = (-1)^{n_{14}+n_{34}-n_{13}-n_{23}-n_{33}-n_{12}-n_{22}-n_{11}}\,|g.\text{conjugate}()\rangle$ (`gelfand::conjugate_phase()`).
- **Physical-to-canonical transformation:** A physical state $|R\,\zeta\,S\,M_S\,T\,M_T\rangle$ ($R = (P_1, P_2, P_3)$) is expanded in canonical Gelfand states. The highest-weight state $|R\,\zeta\,S\,S\,T\,T\rangle$ is the null space of $S_+ = E_{13}{+}E_{24}$ and $T_+ = E_{12}{+}E_{34}$; S_- and T_- fix the lower-M phases within a ζ multiplet. For $P_1 \neq P_3$ the construction preserves ζ; for a self-conjugate irrep ($P_1 = P_3$), conjugation acts as a projection-independent symmetric matrix D ($D^2 = \mathbb{1}$) that may mix ζ (e.g. $(P_1, P_2, P_3) = (1, 1, 1)$, $S = T = 1$).
- **Generator reduced matrix element factor:** `RME_S`/`RME_T`/`RME_U` carry an extra $\sqrt{(2S'{+}1)(2T'{+}1)}$ relative to the Wigner–Eckart reduced element.
- **Assertion checks.** Invalid labels or out-of-range indices abort via assertion.

IV. PYTHON BINDINGS

The `u4lib` extension module reproduces the C++ API one-to-one. Build it with `make python` using the interpreter whose ABI matches the extension, and keep `u3lib`'s `python/` on the import path, since `u4lib` depends on it. The irreps (`su4.irrep`, `irrep`), `gelfand`, `physicalvector`, the free functions `Eta`, `Zeta`, `decomposition`, `gelfandBasis`, `physicalBasis`, the generator RMEs `RME_S`/`RME_T`/`RME_U`, the transformations `phys_to_cano`/`phys_to_cano_allM`, and the five database accessors all carry the names and semantics of Sec. III. Each structured container supports `__call__` for multiplicity-indexed access, exposes its flat `data()` as a NumPy array, and prints itself; querying a database before `initialize()` or after `finalize()` raises `RuntimeError` instead of aborting. The `U4U9` container, only a scratch-buffer view in C++, is copied automatically so that the Python object owns its data.

```
import u4lib as u4
u4.initialize()
try:
    blk = u4.wigU4SU2SU2db().get_U4SU2SU2(1,0,0,1,1, 1,0,0,1,1, 2,0,0,2,2)
    x = blk(0, 0, 0, 0)                  # (zetaA,zetaB,zetaC,eta) = 1
finally:
    u4.finalize()
```

The examples in the Jupyter notebook `python/tutorial.ipynb` cover irreps, bases, coupling, and recoupling. Consult `python/BUILD_NOTES.md` for ABI and toolchain requirements.

V. AI-READY INTERFACE

A. Skill catalog and orchestration

A request such as "couple these spin-isospin states" leaves several details for a program to resolve: the representation convention, doubled quantum numbers, multiplicity order, and the appropriate database. The skills in `agents/` make those choices explicit. The index `agents/AGENTS.md` points to shared runtime instructions and individual skills, each with an input specification, a library call, convention notes, and a numerical check.

An assistant uses the catalog to assemble and execute a calculation on the local installation. It reports the library's output with enough labels to identify the states and coupling channels, together with the check performed. The coefficient evaluation remains in the compiled C++ or Python routines; the assistant is not a substitute for their numerical algorithms. It must resolve an ambiguous request or explain an undefined quantity before proceeding. One example is a request for a first-order Casimir of an SU(4) irrep. The numerical library runs locally and needs no network service.

The requests in Table II illustrate the catalog's coverage. Their expected values were verified using the library. Spin and isospin inputs use the doubled labels $SS = 2S$ and $TT = 2T$.

B. Evaluation objectives

The benchmark tests whether the catalog helps an agent turn a U(4) calculation into a working program. Correctness is assessed separately from the resources spent reaching a solution. We examine two expectations:

1. Resource efficiency: Ready access to argument conventions and usage examples should reduce source-code searches and unsuccessful implementation attempts, lowering token use, runtime, and API cost.

2. Dependence on task complexity: Guidance about memory ownership and thread safety should become more useful when the task moves beyond Python bindings to C++ storage management and parallel execution.

C. Experimental methodology

1. Benchmark Tasks

The three tasks exercise different parts of the interface and place progressively greater demands on storage and concurrency:

Skill	Example prompt (expected result)
`dim`	"Dimension of the SU(4) irrep $(1,0,1)$ – equivalently the U(4) irrep $[2,1,1,0]$?" (15, the adjoint; shift-invariant)
`casimirs`	"C_2, C_3, C_4 for the SU(4) adjoint $(1,0,1)$, and C_1, C_2, C_3, C_4 for the U(4) irrep $[2,1,1,0]$." (SU(4): 16, 0, 72; U(4): C_1=4, 16, 0, 72 – C_1 is U(4)-only)
`cano-list`	"Enumerate the canonical U(4) Gelfand basis of $(1,0,0)$ with each state's weights and subgroup labels." (4 states = dim)
`phys-list`	"List the physical spin-isospin basis (ζ, S, T) of $(2,0,0)$ and verify the dimension sum rule." ((S,T)=(1, 1), (0, 0), each ζ=1; $\sum \zeta(2S{+}1)(2T{+}1)$=10)
`cano-generators`	"Act with E_{12} on the Gelfand state $\lvert 2,0,0,0;2,0,0;2,0;1\rangle$ of $(2,0,0)$; give the resulting state and matrix element." ($\lvert 2,0,0,0;2,0,0;2,0;2\rangle$ with $\sqrt{2}$)
`phys-generators`	"Compute $\langle (2,0,0)\,\zeta{=}1\,S{=}1\,T{=}1 \lVert \mathbf{S} \rVert (2,0,0)\,\zeta{=}1\,S{=}1\,T{=}1\rangle$ and $\langle \lVert \mathbf{U} \rVert \rangle$ on the same state." ($\sqrt{18} \approx 4.243$ and 3)
`phys-to-cano`	"Express $\lvert (2,0,0)\,\zeta{=}1\,S{=}1\,M_S{=}1\,T{=}1\,M_T{=}1\rangle$ in the canonical Gelfand basis." (single state $\lvert 2,0,0,0;2,0,0;2,0;2\rangle$, amplitude 1)
`inner-mult`	"Spin-isospin multiplicity ζ of (S,T)=(1, 1) in the adjoint $(1,0,1)$?" (1)
`outer-mult`	"Outer multiplicity η of $(1,0,1)$ in $(1,0,1) \otimes (1,0,1)$?" (2)
`decomposition`	"Full decomposition of $(1,0,0) \otimes (1,0,0)$ with outer multiplicities; check the dimension sum rule." ($(2,0,0){+}(0,1,0)$, $\sum \eta \cdot \dim = 16$)
`cano-coupling`	"SU(4)⊃U(3) reduced coupling for highest-weight states $[1,0,0,0]\,[1,0,0] \times [1,0,0,0]\,[1,0,0] \to [2,0,0,0]\,[2,0,0]$ of $(1,0,0) \times (1,0,0) \to (2,0,0)$; verify it equals 1."
`phys-coupling`	"Spin-isospin coupling $(1,0,0)\,\zeta{=}1\,S{=}\frac{1}{2}\,T{=}\frac{1}{2} \times (1,0,0)\,\zeta{=}1\,S{=}\frac{1}{2}\,T{=}\frac{1}{2} \to (2,0,0)\,\zeta{=}1\,S{=}1\,T{=}1$; verify it equals 1."
`u6recoupling`	"U-recoupling for $(1,0,0) \otimes (1,0,0) \otimes (1,0,0) \to (3,0,0)$ through intermediate irreps $AB = (2,0,0)$ and $BC = (2,0,0)$." (1)
`z6recoupling`	"Z-recoupling between $(AB)C$ and $(AC)B$ routes for $(1,0,0) \otimes (1,0,0) \otimes (1,0,0) \to (3,0,0)$ with intermediate irreps $AB = (2,0,0)$ and $AC = (2,0,0)$." (1)
`u9recoupling`	"9-coefficient recoupling for four fundamentals $(1,0,0)$ coupling to $(4,0,0)$ with intermediate irreps $AB = CD = AC = BD = (2,0,0)$." (1)
`conjugation`	"Conjugate the SU(4) irrep $(2,0,0)$ and a Gelfand state of it, reporting the phases." (irrep $\to (0,0,2)$, phase power 3)

TABLE II: The `u4lib` agent skills with a representative prompt for each.

- Task 1: Python Racah Recoupling Engine (`consumer/recouple.py`). Write a Python utility using the pybind11 bindings to evaluate $U(4)$ 6-$(\lambda_1, \lambda_2, \lambda_3)$ Racah recoupling coefficients $U(p_1, p_2, p, p_3; p_{12}, p_{23})$ and Z-recoupling coefficients via `u4lib.u6recoupling` and `u4lib.z6recoupling`. The implementation must check the unitary symmetry relations among fundamental representations (for instance, $(1,0,0) \otimes (1,0,0) \otimes (1,0,0) \to (3,0,0)$) and export structured numerical outputs. *Architectural complexity: Low* (declarative Python bindings, automatic garbage collection, single-threaded execution).

- Task 2: C++ 9-Coefficient Batch Exporter (`consumer/u9_batch.cpp`). Build a C++ batch exporter for four-irrep U(4) 9-recoupling blocks via `u4::u4u9db()`. The program must parse 27-integer input specifications (Dynkin triples for 9 participating irreps), compute intermediate outer multiplicities η, retain coefficient buffers across requests without aliasing, configure build dependencies, and serialize block and entry records. *Architectural complexity: Medium* (C++ typing, header inclusion, manual workspace lifecycle management, Dynkin and partition label handling).

- Task 3: C++ OpenMP Multi-threaded Concurrency (`consumer/coupling_batch.cpp`). Implement parallel canonical and physical coupling evaluations with OpenMP directives (`#pragma omp parallel for`). The implementation needs thread-private workspaces for the null-space solver, the required compiler flags (`-fopenmp`), and scalable multi-core execution. *Architectural complexity: High* (shared-memory concurrency, thread-private state isolation, race-hazard avoidance).

2. Workspace preparation

Runs started in independent temporary directories (`/tmp/u4lib-bench-...`), each with its own consumer build environment. Files and build products from earlier trials were not reused. The workspace snapshot determined which documentation the agent could consult:

- Control (standard prompts, zero skills): Source files, C++ headers, and build scripts were available, but the `agents/` and `.gemini/` skill directories were removed.
- Treatment (domain skills): The workspace also contained the repository's skill catalog (`agents/skills/`), indexed by `agents/AGENTS.md`. The prompt instructed the agent to consult it for conventions and usage patterns.

3. Completion and validation

Agents could read and edit files, inspect directories, and run shell commands without intervention. A run ended when the agent declared completion or used 50 steps. The workspace was then frozen for evaluation against the hidden tests in `fixtures-main.json`.

The grader compared numerical output with exact analytic coupling and recoupling values for several irreps, allowing a tolerance of 10^{-12}. We checked the grader itself with `validate_grader.py` before the experiment. Reference implementations were modified to include phase inversions, memory leaks, uninitialized reads, and race conditions. All such validation cases were classified correctly: sensitivity and specificity were 100%, with no false positives or negatives.

4. Model and Statistical Protocol

Every trial used Anthropic's Claude 3.5 Sonnet (`claude-3-5-sonnet-20241022`) in the same autonomous test environment. Each of the three tasks contributed $N = 10$ matched Control/Treatment pairs, giving 60 runs overall.

For every run, four primary metrics were recorded:

1. Functional Pass Rate (%): Percentage of runs passing all hidden unit tests.
2. Total Token Consumption: Cumulative sum of prompt tokens, cache-read tokens, and completion tokens consumed across all agent steps.
3. Wall-Clock Latency (s): Total elapsed real time from task dispatch to final agent termination, measured externally by the evaluation harness.
4. Monetary Cost (USD): Total API billing incurred, computed from official token pricing.

Differences were assessed using two-tailed Welch's t-tests (unequal variance assumption), reporting t-statistics, degrees of freedom, and p-values.

D. Results and discussion

1. Accuracy and resource use

All Treatment programs passed the hidden tests; Control programs failed in four runs. Resource use was also lower with the catalog. The totals for the 30 matched pairs are reported below, with task-level results in Table III and Fig. 1:

- Token Consumption: with the skill catalog, mean token consumption dropped from $717,199 \pm 257,789$ to $307,305 \pm 84,146$ tokens, a 57.1% aggregate reduction ($t = -8.28$, $df = 35.11$, $p = 9.05 \times 10^{-10}$, Welch's t-test).
- Execution Latency: mean wall-clock time shortened from 124.21 ± 53.90 s to 69.83 ± 29.41 s, a 43.8% aggregate speedup ($t = -4.85$, $df = 44.86$, $p = 1.52 \times 10^{-5}$).

TABLE III: Performance of Control (standard prompts, zero skills) versus Treatment (repository-local domain skills) on `u4lib` with Claude 3.5 Sonnet, over $N = 10$ matched pairs per task (60 runs in total). Token counts, runtimes, and costs are given as Mean ± Standard Deviation, and significance follows from two-tailed Welch's t-tests.

Task	Condition	Pass Rate	Total Tokens	Time (s)	Cost ($)	Token Δ	Time Δ
Task 1: Python	Control (N=10)	90.0% (9/10)	$650,386 \pm 304,782$	97.35 ± 42.34	\$0.3949	—	—
	Treatment (N=10)	100% (10/10)	$318,884 \pm 109,257$	62.04 ± 42.67	\$0.2377	**−51.0%****	**−36.3%**
Task 2: C++ Batch	Control (N=10)	90.0% (9/10)	$693,475 \pm 230,556$	117.69 ± 24.53	\$0.4058	—	—
	Treatment (N=10)	100% (10/10)	$311,188 \pm 75,098$	77.24 ± 25.88	\$0.2446	**−55.1%*****	**−34.4%****
Task 3: C++ OpenMP	Control (N=10)	80.0% (8/10)	$807,737 \pm 231,065$	157.59 ± 69.85	\$0.4628	—	—
	Treatment (N=10)	100% (10/10)	$291,843 \pm 69,332$	70.20 ± 12.99	\$0.2443	**−63.9%*****	**−55.4%****
Grand Aggregate	Control (N=30)	86.7% (26/30)	$717,199 \pm 257,789$	124.21 ± 53.90	\$0.4212	—	—
	Treatment (N=30)	100% (30/30)	$307,305 \pm 84,146$	69.83 ± 29.41	\$0.2422	**−57.1%*****	**−43.8%*****

$^{**}p < 0.01$ (Task 1: $t_{\text{tok}} = -3.24$, p=0.0077; Task 2 time: $t = -3.59$, p=0.0021; Task 3 time: $t = -3.89$, p=0.0032);

$^{***}p < 0.001$ (Task 2 tok: $t = -4.99$, $p = 4.2 \times 10^{-4}$; Task 3 tok: $t = -6.76$, $p = 3.7 \times 10^{-5}$);

Grand Aggregate: $t_{\text{tok}} = -8.28$, $t_{\text{time}} = -4.85$; Welch's two-tailed t-test.

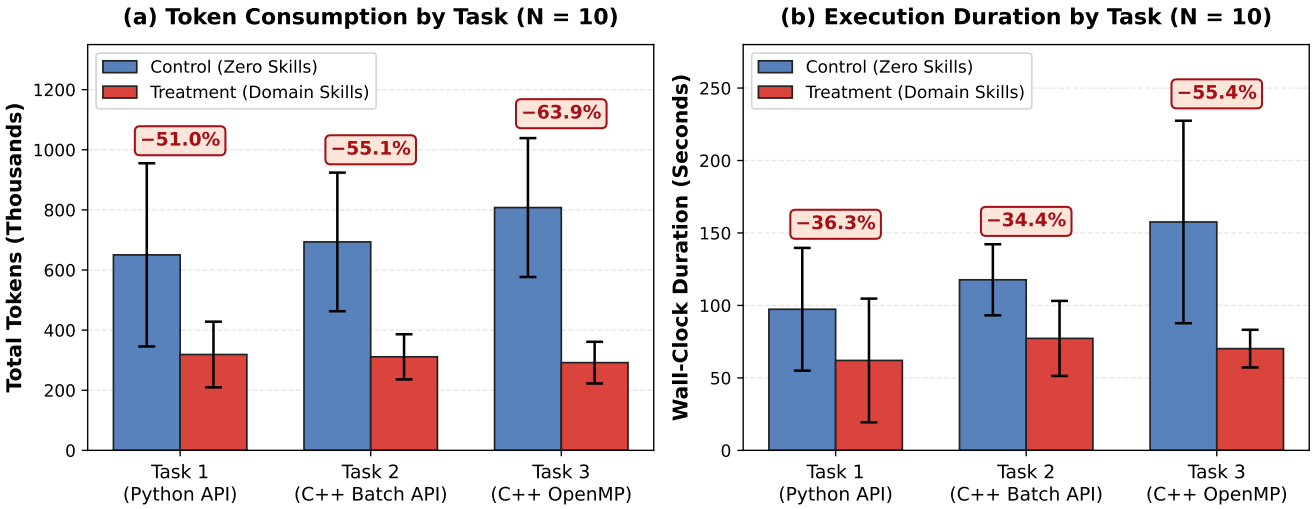


FIG. 1: Autonomous-agent performance under Control (standard prompts, zero skills) and Treatment (repository-local domain skills) on `u4lib`, over $N = 10$ matched pairs per task (60 runs total). (a) Per-task token consumption (thousands), with reductions of 51.0%, 55.1%, and 63.9%. (b) Per-task wall-clock runtime (seconds), with speedups of 36.3%, 34.4%, and 55.4%. Error bars show ±1 sample standard deviation.

- API Cost: average per-task inference cost went from \$0.4212 to \$0.2422 (−42.5%, $t = -9.52$, $df = 35.64$, $p = 2.49 \times 10^{-11}$).
- Reliability: all 30 Treatment runs passed (10/10 on each of the three tasks), whereas Control runs failed four times across the suite (9/10, 9/10, and 8/10 on Tasks 1–3), an aggregate pass rate of 86.7%.

2. *Dependence on task complexity*

Across the three tasks, the relative token savings grow with the intended rise in complexity, while the execution-time savings peak on the parallel C++ task without following a strictly monotonic trend:

$$\Delta_{\text{tokens}} = -51.0\% \text{ (Task 1)} \longrightarrow -55.1\% \text{ (Task 2)} \longrightarrow -63.9\% \text{ (Task 3)},$$
$$\Delta_{\text{time}} = -36.3\% \text{ (Task 1)} \longrightarrow -34.4\% \text{ (Task 2)} \longrightarrow -55.4\% \text{ (Task 3)}.$$

Token savings increased from the Python task to the parallel C++ task. Runtime savings did not increase at every step, although Task 3 had the largest reduction in both measures. This is consistent with a benefit from documenting

implementation constraints, but three tasks are too few to establish how that benefit scales. The run logs help explain the differences observed here.

3. *Errors observed in the coding runs*

The logs identify three recurring sources of difficulty:

1. Task 1 (argument and channel order): Control agents searched headers and tried different signatures to determine how the Python interface represents $U(4)$ labels and distinguishes U- from Z-recoupling. In one Control run (`task1-control-8`), the agent inverted intermediate coupling channels, producing sign errors that failed validation. Treatment agents used the skill catalog's canonical argument order and usage patterns, completing the task in an average of $318,884$ tokens (a 51.0% reduction, $p = 0.0077$) with zero failures.

2. Task 2 (representation labels and retained buffers): In Task 2, C++ 9-coefficient recoupling requires evaluating four-irrep blocks via `u4::u4u9db()` and retaining all blocks in memory before serialization. Control agents had difficulty distinguishing 3-component SU(4) Dynkin highest-weight labels (P_1, P_2, P_3) and 4-component U(4) partition labels $(n_{14}, n_{24}, n_{34}, n_{44})$. In `task2-control-2`, the unguided agent mistakenly passed Dynkin triples into the 4-label partition constructor with $n_{44} = 0$, violating internal partition betweenness invariants and triggering a fatal assertion failure (`Assertion failed: to_uint8_t, file Int.h, line 57`). The logs also show repeated scratch-buffer aliasing errors. Treatment agents used the catalog's constructor examples and storage instructions, with a 55.1% token reduction ($p = 4.2 \times 10^{-4}$) and a 100% pass rate.

3. Task 3 (shared solver state): Parallel $U(4)$ coupling requires care because the null-space solver modifies scratch memory during factorization. Control implementations shared database or solver instances across threads in `#pragma omp parallel for` blocks. This produced race conditions, memory corruption, and non-deterministic crashes, causing two runs to fail (`task3-control-2` and `task3-control-5`). Even successful Control runs required repeated debugging (averaging 157.59 s and $807,737$ tokens). Treatment implementations followed the catalog's OpenMP rules and allocated thread-private workspaces. They produced deterministic output in 70.20 s and $291,843$ tokens—a 63.9% token reduction ($p = 3.7 \times 10^{-5}$) and 55.4% speedup ($p = 0.0032$).

4. *Implications for scientific software*

For this U(4) benchmark, the most useful guidance concerned the boundary between mathematical notation and executable code: choosing the right irrep constructor, ordering coupling channels, and preserving coefficient storage. Parallel execution added a separate requirement to isolate solver state. Documenting these details in the skills was associated with 57.1% lower aggregate token use and 43.8% shorter runtime. All 30 Treatment runs passed, compared with 26 Control runs; two Control failures occurred in the parallel C++ task. These results describe one model and three tasks. Testing other models and calculation types is necessary before drawing broader conclusions about reliability or resource savings.

VI. BUILD AND INSTALLATION

Install `u3lib` before configuring `u4lib`. The U(4) headers in `include/` can then be included directly in user programs. Source code is available at https://github.com/Deep-SANE/u4lib.

Run `make config` or `./config/configure.sh` to locate `u3lib` and resolve compiler flags and dependency paths. Configuration is written to `config/config.generated.mk`; no manual environment variables are required. Use `make` for the example CLI programs in `bin/`, `make test` to rebuild and execute the regression suite, and `make python` for the pybind11 extension `python/u4lib.so`.

The tests cover irrep dimensions, Gelfand structure, SU(2) × SU(2) branching, coupling orthonormality in ζ and η, conjugation identities, and analytic reference values. Numerical results are compared within tolerances.

All core external dependencies are provided by `u3lib`:

- `u3lib` (prerequisite): provides the base group-theoretic infrastructure, shared utilities, and the libwignernj SU(2) Wigner backend.

- Null-space solver (selected via `NULLSPACE_SOLVER=qr|sns`, default `qr`):

  - Dense QR (default): dense QR null-space solver via Eigen3; requires no external sparse solver.
  - `sns` (https://gitlab.fit.cvut.cz/krastale/dp-sparse-nullspace): sparse null-space solver enabled via `USE_SNS=1`.

- Eigen3, Boost (headers), OpenMP, and fmt: inherited from `u3lib`.
- pybind11 (Python bindings): https://github.com/pybind/pybind11.

`u4lib` ships as an independent, self-contained repository with a uniform `make config`/`make`/`make test`/`make install` interface and no required environment variables.

Acknowledgments

This work is dedicated to the memory of J. P. Draayer, who passed on September 13, 2026 and whose pioneering work in algebraic computational nuclear physics — from Elliott's SU(3) model and the Symplectic Sp(3, R) model to the *ab initio* Symmetry-Adapted No-Core Shell Model and, most recently, Wigner's SU(4) supermultiplet model — inspired this framework. This library implements the U(4) coupling and recoupling procedure of Ref. [1]. The authors also gratefully acknowledge F. Pan and D. Kekejian for valuable discussions. PD was supported by the Institute of International Education through the Quad Fellowship, Louisiana State University, and Quantum CodeX. DL and TD were funded by Quantum CodeX.

[1] P. Dang, J. P. Draayer, F. Pan, T. Dytrych, D. Langr, D. Kekejian, K. S. Becker, and N. Thompson, "Coupling and Recoupling Coefficients for Wigner's U(4) Supermultiplet Symmetry," Eur. Phys. J. Plus **139**, 933 (2024), doi:10.1140/epjp/s13360-024-05581-6.
[2] E. P. Wigner, "On the Consequences of the Symmetry of the Nuclear Hamiltonian on the Spectroscopy of Nuclei," Phys. Rev. **51**, 106 (1937), doi:10.1103/PhysRev.51.106.
[3] J. P. Elliott, "Collective motion in the nuclear shell model. I. Classification schemes for states of mixed configurations," Proc. R. Soc. A **245**, 128 (1958), doi:10.1098/rspa.1958.0072.
[4] J. P. Elliott, "Collective Motion in the Nuclear Shell Model. II. The Introduction of Intrinsic Wave-Functions," Proc. R. Soc. A **245**, 562 (1958), doi:10.1098/rspa.1958.0101.
[5] T. Dytrych *et al.*, "Collective Modes in Light Nuclei from First Principles," Phys. Rev. Lett. **111**, 252501 (2013), doi:10.1103/PhysRevLett.111.252501.
[6] T. Dytrych *et al.*, "Physics of Nuclei: Key Role of an Emergent Symmetry," Phys. Rev. Lett. **124**, 042501 (2020), doi:10.1103/PhysRevLett.124.042501.
[7] Yu. S. Lutostansky and V. N. Tikhonov, "Charge-exchange resonances and restoration of the Wigner SU(4)-symmetry in heavy and superheavy nuclei," EPJ Web Conf. **107**, 06004 (2016), doi:10.1051/epjconf/201610706004.
[8] K. T. Hecht and S. P. Pang, "On the Wigner Supermultiplet Scheme," J. Math. Phys. **10**, 1571 (1969), doi:10.1063/1.1665007.
[9] J. P. Draayer, "U(4) $\supset$ SU(2) $\otimes$ SU(2) Projection Techniques," J. Math. Phys. **11**, 3225 (1970), doi:10.1063/1.1665119.
[10] A. Partensky and C. Maguin, "The SU(4) $\supset$ SU(2) $\otimes$ SU(2) chain," J. Math. Phys. **19**, 511 (1978), doi:10.1063/1.523687.
[11] D. J. Rowe and J. Repka, "The Representations and Coupling Coefficients of su(n); Application to su(4)," Found. Phys. **27**, 1179 (1997), doi:10.1007/bf02551440.
[12] F. Pan, Y. Wu, A. Li, Y. Zhang, L. Dai, and J. P. Draayer, "An algebraic projection procedure for construction of the basis vectors of irreducible representations of U(4) in the $\mathrm{SU_S}(2) \otimes \mathrm{SU_T}(2)$ basis," Eur. Phys. J. Plus **138**, 662 (2023), doi:10.1140/epjp/s13360-023-04261-1.
[13] F. Pan, L. Dai, and J. P. Draayer, "Wigner coefficients of U(4) $\supset \mathrm{SU_S}(2) \otimes \mathrm{SU_T}(2)$," Nucl. Phys. A **1040**, 122746 (2023), doi:10.1016/j.nuclphysa.2023.122746.
[14] F. Pan, L. Dai, and J. P. Draayer, "Elementary coupling coefficients for the Wigner supermultiplet symmetry," Comput. Phys. Commun. **296**, 109025 (2024), doi:10.1016/j.cpc.2023.109025.
[15] A. Alex, M. Kalus, A. Huckleberry, and J. von Delft, "A numerical algorithm for the explicit calculation of SU(N) and SL(N, $\mathbb{C}$) Clebsch–Gordan coefficients," J. Math. Phys. **52**, 023507 (2011), doi:10.1063/1.3521562.
[16] M. Kuhn and H. Walliser, "Program for calculating SU(4) Clebsch–Gordan coefficients," Comput. Phys. Commun. **179**, 733 (2008), doi:10.1016/j.cpc.2008.06.009.
[17] J. P. Draayer and Y. Akiyama, "Wigner and Racah coefficients for SU_3," J. Math. Phys. **14**, 1904 (1973), doi:10.1063/1.1666267.
[18] Y. Akiyama and J. P. Draayer, "A User's Guide to Fortran Programs for Wigner and Racah Coefficients of SU_3," Comput. Phys. Commun. **5**, 405 (1973), doi:10.1016/0010-4655(73)90077-5.
[19] C. Bahri, D. J. Rowe, and J. P. Draayer, "Programs for generating Clebsch–Gordan coefficients of SU(3) in SU(2) and SO(3) bases," Comput. Phys. Commun. **159**, 121 (2004), doi:10.1016/j.cpc.2004.01.005.

[20] T. Dytrych, D. Langr, J. P. Draayer, K. D. Launey, and D. Gazda, "SU3lib: A C++ library for accurate computation of Wigner and Racah coefficients of SU(3)," Comput. Phys. Commun. **269**, 108137 (2021), doi:10.1016/j.cpc.2021.108137.
[21] P. Dang, J. P. Draayer, F. Pan, and K. S. Becker, "New Procedure for Evaluation of U(3) Coupling and Recoupling Coefficients," Eur. Phys. J. Plus **139**, 704 (2024), doi:10.1140/epjp/s13360-024-05501-8.
[22] D. B. Kaplan and M. J. Savage, "The spin-flavor dependence of nuclear forces from large-N QCD," Phys. Lett. B **365**, 244 (1996), doi:10.1016/0370-2693(95)01277-X.
[23] B.-N. Lu *et al.*, "Essential elements for nuclear binding," Phys. Lett. B **797**, 134863 (2019), doi:10.1016/j.physletb.2019.134863.
[24] J. Zhang, J. Vala, S. Sastry, and K. B. Whaley, "Geometric theory of nonlocal two-qubit operations," Phys. Rev. A **67**, 042313 (2003), doi:10.1103/PhysRevA.67.042313.
[25] A. V. Gorshkov, M. Hermele, V. Gurarie, C. Xu, P. S. Julienne, J. Ye, P. Zoller, E. Demler, M. D. Lukin, and A. M. Rey, "Two-orbital SU(N) magnetism with ultracold alkaline-earth atoms," Nat. Phys. **6**, 289 (2010), doi:10.1038/nphys1535.
[26] A. F. Young *et al.*, "Spin and valley quantum Hall ferromagnetism in graphene," Nat. Phys. **8**, 550 (2012), doi:10.1038/nphys2307.